Programs for a Frozen Machine: The Description Length of Language and Task in Optimized Prefixes

Seminar report, Mechanistically Interpreting Multilingual Language Models (WT 2025/26)

Georgii Sirotenko

October 2026

1 Abstract

A frozen language model is a fixed machine; a soft prefix that makes it perform a task is a program for that machine, and the shortest such program is a model-relative description length of the task. This report measures that length for a frozen Qwen2.5-1.5B base model over a grid of three tasks (translation from English, factual recall, sentence continuation) and three output languages (Russian, German, English), in two units: prefix slots and variational bits. Four results. (i) Slots are not a description length: translation saturates at one slot, a single trained slot beats the natural-language prompt everywhere, and a conditioning string is worth less than one slot. (ii) In bits, a translation program costs about 500 bits at 10% excess loss, a Russian recall program five times that, and English recall most of all; continuation has nothing to compress. (iii) Task is the shared factor and language the cheap modifier: coding all eight cells with one centre per task and language deviations costs 11% less than eight independent programs, coding them with one centre per language costs 40% more; a target language costs 500–700 bits given the translation task; recall programs are language-specific, do not compose zero-shot, and are read deep in the network, whereas translation programs are read almost entirely in the first block. (iv) The variational objective selects compact programs but the standard optimiser does not find them: a conservative learning rate and checkpoint selection on the full objective lower the measured frontier by 20–70% with fewer runs, which turns the open problem of Shaw et al. (2026) into a practical recipe at this scale.

2 Question and framing

For a frozen model M, task t and output language l, let $K_M(t, l)$ be the shortest soft prefix that performs t in l at a chosen distortion. The project asks how $K_M(t, l)$ is distributed between language and task, whether the two parts are separable, how far apart the programs for different cells lie, and where in depth the model reads them. All lengths are relative to one model, tokenizer and format; the claims are orderings, differences and ratios within that model.

Two units are used throughout. **Slots** are the number k of prefix positions, a structural unit of attention. **Bits** are the description length of the prefix under a prior: the prefix is a diagonal-Gaussian posterior and its KL divergence to a prior fitted to the token-embedding matrix is its code length under the bits-back argument (Hinton and van Camp 1993; Blier and Ollivier 2018). This is the measurement used by Shaw et al. (2026), who treat prompt tokens as a program tape and the Transformer as the interpreter. Counting tokens, the convention of prompt tuning (Lester et al. 2021), is a capacity measure, and Section 4.1 shows why it fails as a description length.

Distortion is the teacher-forced code length L of the target in bits per target token, normalised per cell as $\hat{D} = (L - L_{\text{floor}})/(L_{\text{none}} - L_{\text{floor}})$, with L_{none} the no-prefix loss and L_{floor} the better of a per-cell LoRA fine-tune and the best prefix run. The operating points $\hat{D} \leq 0.25$ and $\hat{D} \leq 0.10$ were preregistered from a

dev pilot; $\hat{D} \leq 0.05$ is reported as exploratory.

3 Setup

Model and data. `Qwen/Qwen2.5-1.5B` base, frozen, bfloat16 on an A100. Translation and continuation from FLORES+ (dev split for training and selection, devtest for reporting); recall from mLAMA (Kassner et al. 2021), with an English cloze prefix and the object rendered in the target language, split by uuid. Translation into English is excluded, so the grid has eight cells.

Designs. *Design A*: $k \in \{1, 2, 4, 8, 16, 32\}$ free slot vectors, two seeds, 1500 steps, no penalty; repeated with the target language given, with the task given, and the natural-language prompt with no prefix. *Flat variational*: $k \in \{2, 4, 8\}$, $\beta \in \{0.3, 1, 3\}$, one KL to the global prior. *Hierarchical*: one centre per task with a deviation per language, and the reverse nesting, $k \in \{2, 4, 8\}$, $\beta \in \{0.3, 1, 3\}$, deviation prior $\mathcal{N}(\mu_{\text{centre}}, \tau^2\sigma_0^2 I)$ with $\tau = 1$ in units of the prior σ_0. *Composition*: tied task parts and language parts, six parts for eight cells, with five hold-out sets. *Distance*: warm-start prequential learning cost from every cell's $k = 4$ prefix toward every cell over 512 examples. *Regime 2*: per-example input vectors, $m \in \{1, 2, 4, 8\}$, with and without the task prefix. *Mechanism*: the prefix positions' hidden states from a trained prefix patched into one block's attention input of a run with a random prefix, for every block.

Two variational protocols. The preregistered protocol (`flat`) used learning rate 0.05, the checkpoint with the lowest dev distortion at the posterior mean, and posterior σ initialised at five times the prior. Its histories (Section 4.8) showed that this protocol overshoots the rate and selects on the wrong statistic. A corrected protocol (`flat2`) uses learning rate 0.01, dev distortion scored on posterior samples, the checkpoint with the lowest dev objective (distortion plus β times the rate), and σ initialised at the prior. `flat2` was run on the five measurable cells at $k \in \{2, 4\}$, $\beta \in \{1, 3\}$ and is the headline where it exists; `flat` is reported alongside. The hierarchical runs use the corrected σ initialisation but the preregistered selection rule.

Inventory. 273 Design A, baseline and floor rows; 72 flat and 20 flat2 rows; 144 hierarchical rows (54 joint runs); 72 composition rows; 72 distance rows; 32 regime 2 rows; 1,120 mechanism rows. About 32 A100 hours. Every number below is recomputed by `notebooks/01_figures.ipynb` from the CSVs.

4 Results

4.1 Slots are not a description length (H1, slot unit; H2, slot unit)

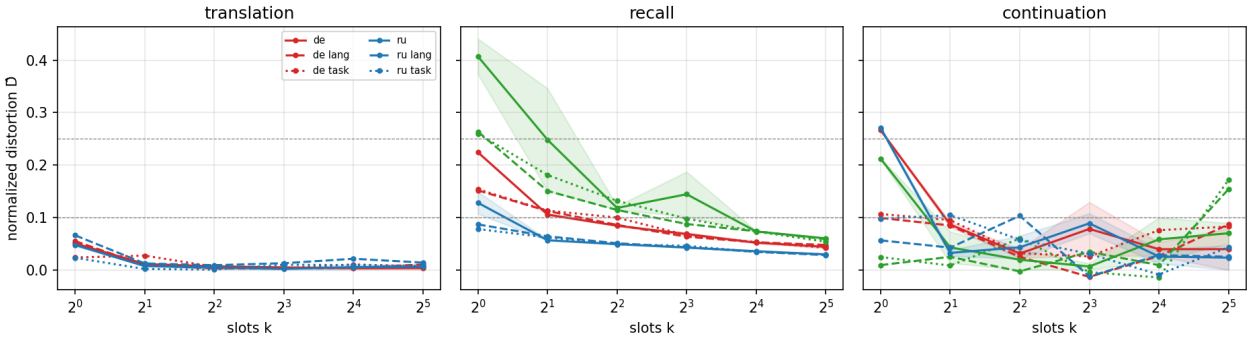


Figure 1: $\hat{D}$ against the number of slots k, Design A, seed means with the seed range shaded; dashed lines are the operating points; line style is the conditioning format.

Translation reaches $\hat{D} \leq 0.10$ with one slot in every language and conditioning, and its curve is flat from there. Recall keeps improving to $k = 32$ with the budget binding (a 3000-step run on recall-ru at $k = 8$ moves $\hat{D}$ from 0.043 to 0.032 and is still falling). Continuation is noise: its floor lies within 0.05–0.15 bits per

token of the no-prefix loss, so the denominator of $\hat{D}$ is below the seed variation, and continuation is excluded from every headline claim. Seed gaps are below 0.011 on translation, 0.04 on recall-ru and recall-de, and 0.20 on recall-en, the one cell where the slot curve is not reliable.

Table 1 gives k^*, the smallest k reaching each level.

cell	$k^*(0.25)$	$k^*(0.10)$	$k^*(0.05)$, expl.
translation-ru	1	1	1
translation-de	1	1	2
recall-ru	1	2	4
recall-de	1	4	32
recall-en	2	16	not reached

Two observations make the slot count unusable as a description length. First, the natural-language prompt, scored as a context with no prefix, is far worse than one trained slot: on recall it leaves $\hat{D}$ at 0.91–0.96, one slot at 0.13–0.41; on translation it leaves 0.12–0.20, one slot 0.05. One vector of 1,536 numbers carries more than an instruction, so "one token" is not a unit of information. Second, conditioning saves almost nothing on the slot axis: giving the language or the task changes $k^*(0.10)$ in none of the translation cells and in one recall cell (recall-en, 16 to 8 for both). Δk_{lang} and Δk_{task} averaged over the measurable cells are both 1.8 slots, and the average is carried entirely by recall-en. The slot axis cannot see the difference between telling the model the language and telling it the task; the bit axis can (Section 4.3).

4.2 Description length in bits (H1, bit unit)

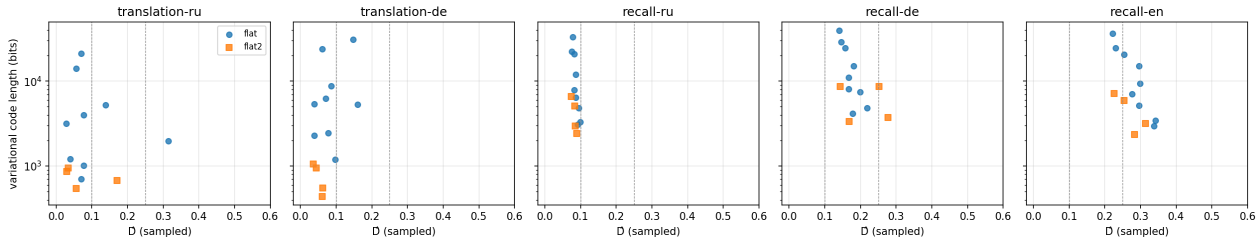


Figure 2: Rate against distortion per cell, every variational run; circles are the preregistered protocol, squares the corrected one. Distortion is scored on posterior samples.

Table 2 is the headline: the smallest code length among runs reaching each operating point, with the preregistered protocol beside the corrected one.

cell	bits at $\hat{D} \le 0.10$ (flat2)	bits at $\hat{D} \le 0.25$ (flat2)	flat2 best $\hat{D}$	flat at $\hat{D} \le 0.25$
translation-de	436	436	0.036	1,194
translation-ru	542	542	0.030	699
recall-ru	2,419	2,419	0.075	3,101
recall-de	not reached	3,343	0.143	4,168
recall-en	not reached	7,155	0.226	24,686

A translation program costs about 500 bits, roughly the content of thirty characters of text under the model. A Russian recall program costs five times that; German and English recall do not reach 10% excess loss within the budget and cost 3,300 and 7,200 bits at 25%. The bootstrap intervals over test examples for the gain of

the $\hat{D} \leq 0.10$ programs over the no-prefix model are narrow: 0.52 ± 0.01 bits per token on translation-ru, 0.50 ± 0.03 on translation-de, 5.69 ± 0.07 on recall-ru.

The ordering across tasks within a language follows the learning gap $L_{\text{none}} - L_{\text{floor}}$ (recall 3.5–6.2 bits per token, translation 0.6–0.9), as H1 expected. The ordering across languages does not follow exposure: English recall is the dearest program, not the cheapest, in slots ($k^*(0.10) = 16$) and in bits, and the result survives restricting all three recall test sets to the 4,212 uuids they share ($\hat{D}$ at $k = 4$: ru 0.049, de 0.079, en 0.122). The English LoRA floor is also the worst of the three (2.58 bits per token against 1.10 for Russian), so what is dear in English is not the language but the facts that mLAMA asks about in English. H1's language part is not supported; the exposure proxy from the companion project was not available and the comparison rests on orderings.

4.3 Task is the shared factor, language the cheap modifier (H2, bit unit)

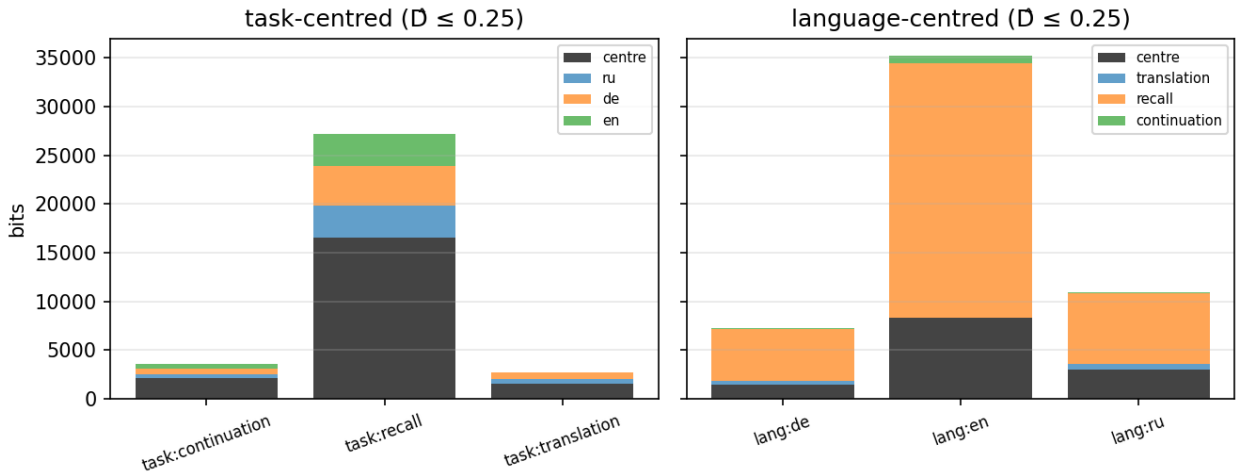


Figure 3: Code length at $\hat{D} \leq 0.25$ under the two hierarchical nestings: the centre is paid once per group, each deviation once per member.

The hierarchical design codes a group of cells as one centre plus one deviation per member. Table 3 compares the three codes for all eight cells at $\hat{D} \leq 0.25$, the only level every cell reaches.

code	bits
one centre per task, language deviations	33,500
eight independent flat programs	37,600
one centre per language, task deviations	53,500

Grouping by task saves 11% overall and almost all of it on recall (27,200 against 32,000 bits for the three recall cells). Grouping by language costs 40% more. The per-member costs explain both numbers. Given the translation centre, a target language costs 485 (ru) or 662 (de) bits, under half a standalone program. Given the recall centre, a language costs 3,300–4,000 bits. In the reverse nesting, given a language centre, continuation costs 100–200 bits, translation 440–630, and recall 5,300 (de), 7,200 (ru) or 26,200 (en) bits. Recall is the program that cannot be shared with anything, and the language-centred code pays for it three times.

For translation alone the hierarchy does not pay: centre plus two deviations costs 2,733 bits at $\hat{D} \leq 0.10$ against 1,893 for two independent programs. The shared centre is dearer than either specialised program, which is the price of a centre that must serve both. H2 holds in bits for recall, where the task given the

language costs two to seven times the language given the task, and is a wash for translation, where both cost about 500 bits.

4.4 Composition (H3)

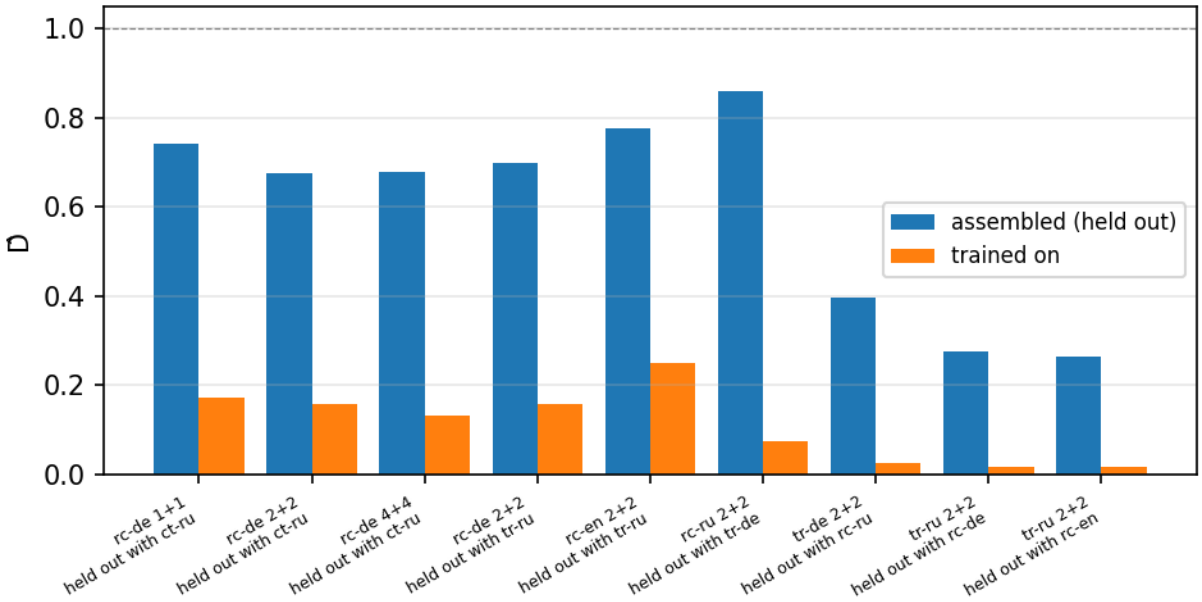


Figure 4: Assembled (held-out) against trained-on $\hat{D}$ of tied task + language parts.

Six tied parts trained on every cell match Design A at the same total slots on translation ($\hat{D}$ 0.017–0.027 against 0.005–0.007 at $2 + 2$) and cost about twice its distortion on recall (0.08–0.25 against 0.05–0.12). Assembled zero-shot for a held-out cell, the parts close 60–73% of the gap between no prefix and the floor on translation (three cells, $\hat{D}$ 0.27–0.40) and 14–32% on recall (four cells, $\hat{D}$ 0.68–0.86). Task and language parts compose partially for translation and barely for recall, consistent with the per-member costs above. The Chinese comparison in the preregistered test was not run; the resource-level part of H3 is untested.

4.5 Distance between programs (H4)

Warm-starting from a prefix of the same task costs 0.85 of training from scratch; from the same language or an unrelated cell, 0.95. The silhouette of the task partition on the symmetrised distance is 0.004 and that of the language partition -0.09: the direction is H4's, the structure is absent. The matrix is one seed, seven pairs per group, and several warm starts cost more than scratch, so it measures optimisation noise as much as distance. The zero-shot block is the strong signal: a prefix applied to any other cell is as bad as no prefix ($\hat{D}$ 0.9–1.0), and a recall prefix applied to continuation is catastrophic ($\hat{D}$ 5–25). The recall program overrides the model's default behaviour rather than adding to it.

4.6 Where the program is read (H5)

Patching all blocks reproduces the donor exactly. Patching block 0 alone recovers 70–87% of the effect for translation and 74–94% for continuation, but only 36–58% for recall, which has secondary peaks at blocks 17 and 25. Single-block fractions sum to about 2.3, so the reading is redundant rather than additive. A translation program is copied into the token stream in the first block and not needed afterwards; a recall program is read repeatedly, deep into the network.



Figure 5: Left: symmetrised warm-start learning cost divided by the from-scratch cost. Right: $\widehat{D}$ of a trained prefix applied unchanged to another cell.



Figure 6: Fraction of the prefix's effect recovered by patching its positions into one block's attention input, mean over cells per task. Fractions are undefined where the donor is not at least 0.1 bits per token better than the recipient, which excludes the deviation probes on translation and continuation.

The preregistered comparison, the block of maximal recovery for the language deviation against the task centre, gives block 0 for every probe on every cell, so it does not separate them. Where the deviation probes are readable, on recall, the language deviation has both early mass (1.0) and late mass (0.5, blocks 14–27), and the task centre is read early. H5's expectation of language early and task in the middle is not what the patches show; what they show is a depth profile that depends on the task, not on which part of the prefix is patched. One detail: the recall centre without its English deviation is worse than a random prefix on English, so the recall centre is a non-English program.

4.7 Input compressibility (H6, exploratory)

With the $k = 4$ task prefix, one per-example vector replaces the input to within 10% of the real-input loss on every cell but translation-ru, which needs two; without the prefix, translation and continuation need two. There is no rate term on the vectors, so this is a slot count and is reported as such. The per-sentence against per-token comparison in H6 was not computed. One oddity for the record: on recall the bare model scores the target better with no input (3.1 bits per token) than with the real query (7.2), which is consistent with confident wrong answers once a subject is named.

4.8 Optimisation: the objective is right, the search is hard

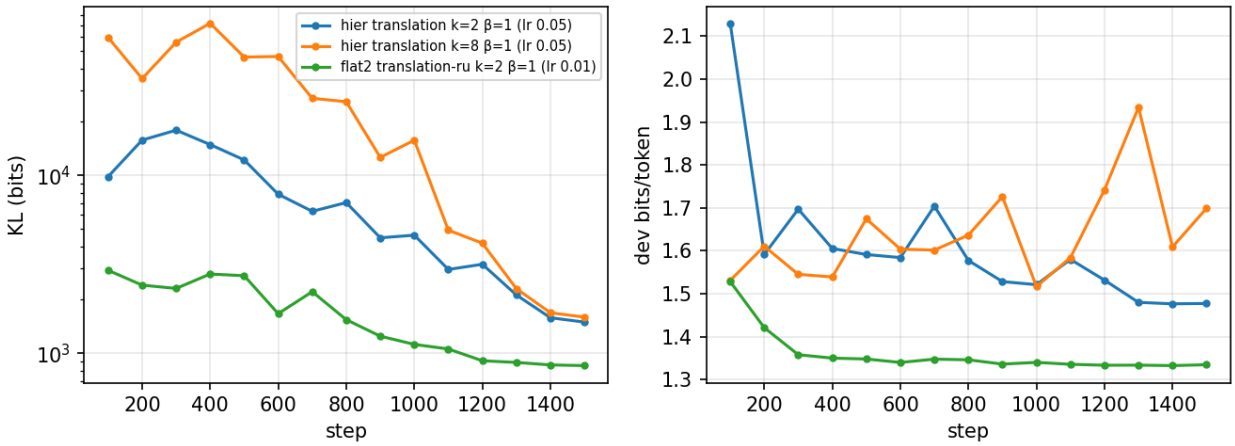


Figure 7: KL and dev loss against training step for two hierarchical runs under the preregistered protocol and one flat2 run.

The saved histories show three things. With learning rate 0.05, two prior standard deviations per Adam step, the KL overshoots in the first hundred steps by an amount that grows with k: about 20,000 bits at $k = 2$ and 110,000–300,000 at $k = 8$, before the cosine schedule lets the KL term pull the means back. At $k = 8$ the final rate is the same as at $k = 2$, but the posterior-mean distortion worsens late, so the distortion-only checkpoint rule selects a step where the rate is still five to ten times higher. And the posterior mean is not the coded object: at $\beta = 3$ the mean scores $\widehat{D}$ 0.35–0.56 while samples score 0.06–0.17, because the model was trained on samples.

The corrected protocol removes all three effects and lowers the measured frontier on every cell, by 22% (translation-ru, recall-ru), 58–63% (translation-de), 20% (recall-de) and 71% (recall-en), with four runs per cell instead of nine, and the rates become monotone in β. Shaw et al. (2026) report that on a parity task the same objective reaches 870 bits from a hand-compiled initialisation but 2.8 million bits from random initialisation, and leave the optimiser as the open problem. At this scale the gap is closed mostly by selecting on the objective and stepping at the scale of the prior.

5 Preregistered comparisons

	comparison	outcome
H1	cost ordering by language exposure and by task difficulty	task ordering supported (recall $\gg$ translation); language ordering reversed on recall (en dearest), robust to common items; exposure proxy unavailable
H2	Δk_{lang} vs Δk_{task}; bit savings under Design B	equal on the slot axis (1.8 slots each, carried by one cell); in bits, task given language costs 2–7× language given task on recall, equal on translation
H3	composition cost at $\hat{D} = 0.25$; Design B minus Design A gap	translation composes partially (60–73% of the gap zero-shot), recall barely (14–32%); hierarchy beats independent programs on recall, not on translation; Chinese not run
H4	silhouette of task vs language clustering	0.004 vs -0.09: task direction, no structure; same-task warm start 0.85 vs 0.95
H5	block of maximal recovery, language deviation vs task centre	block 0 for both; depth profile splits by task instead (recall read deep)
H6	m^* per sentence and per token across languages	one vector suffices with the task prefix; per-token variant not computed; no rate term

6 Deviations and limitations

Three deviations from the preregistration are recorded in the project ledger. The floor is the better of LoRA and the best prefix, because prefixes beat LoRA on five cells. Continuation is excluded from headline claims as unmeasurable. The variational protocol was corrected after the preregistered runs, for the reasons in Section 4.8, and the corrected runs are the headline; both are reported. Two seeds were run instead of three, and the recall budget binds at 1500 steps. The hierarchical runs keep the preregistered selection rule, so their rates carry the selection bias of Section 4.8 and are upper bounds. The distance matrix is one seed. The mechanism probes patch attention inputs at the prefix positions; a residual-stream patch would answer a slightly different question. The τ of the hierarchical design was first run in absolute embedding units, which made the deviation prior forty times wider than the global prior; those runs are kept as a control and not used.

7 What a research version would do

The same code runs the scale axis (Qwen2.5 0.5B to 7B), twenty or more FLORES languages stratified by resource, further tasks, and discrete prefixes. Three changes follow from this report. Select every variational checkpoint on the objective and score distortion on samples. Replace the learning rate by a step size in units of the prior σ. Adopt the per-column Gaussian-mixture prior of Shaw et al. (2026) as a second rate, and charge $\log_2 |k\text{-grid}|$ bits for the choice of k. The hypotheses worth the compute are the ones this report could not settle: the exposure ordering across many languages, the composition cost against resource level, and whether the depth at which a program is read changes with scale.

8 References

- Blier, L. and Ollivier, Y. (2018). The Description Length of Deep Learning Models. NeurIPS.
- Dumas, C. et al. (2025). Separating Tongue from Thought. ACL.
- Hendel, R., Geva, M. and Globerson, A. (2023). In-Context Learning Creates Task Vectors. Findings of EMNLP.
- Hinton, G. E. and van Camp, D. (1993). Keeping the Neural Networks Simple by Minimizing the Description Length of the Weights. COLT.
- Kassner, N., Dufter, P. and Schütze, H. (2021). Multilingual LAMA. EACL.
- Khashabi, D. et al. (2022). Prompt Waywardness. NAACL.
- Lester, B., Al-Rfou, R. and Constant, N. (2021). The Power of Scale for Parameter-Efficient Prompt Tuning. EMNLP.
- Li, X. L. and Liang, P. (2021). Prefix-Tuning. ACL.
- Shaw, P., Cohan, J., Eisenstein, J. and Toutanova, K. (2026). Bridging Kolmogorov Complexity and Deep Learning. ICLR. arXiv:2509.22445.
- Todd, E. et al. (2024). Function Vectors in Large Language Models. ICLR.
- Voita, E. and Titov, I. (2020). Information-Theoretic Probing with Minimum Description Length. EMNLP.
- Vu, T. et al. (2022). SPoT: Better Frozen Model Adaptation through Soft Prompt Transfer. ACL.

9 Appendix: reproduction

Repository `sirotenkogeorgii/prefix-programs`, branch `prefixlab`. `results_sync/` holds every result table, per-example bits, training histories and the prefix checkpoints. `notebooks/01_colab_runner.ipynb` reproduces the runs on an A100 (about 32 hours); `notebooks/01_figures.ipynb` regenerates every figure and table here from the CSVs and `prereg.json` in under a minute; `notebooks/02_live_demo.ipynb` trains one prefix live.